

HTML5 JS API (.NET Core 後端版)

林新德 · shinder.lin@gmail.com

講義版本：2026-09-12

範例專案：<https://github.com/shinder/html5-js-api-examples> 的 `csharp-backend/` 子目錄
(與 Node 範例同一個 repo，共用前端 `public/`)。

目錄

前言	p. 2
適用對象	p. 2
技術棧選擇	p. 2
環境準備	p. 2
與原文件 Express 的最大差異	p. 5
8. Server-Sent Event (SSE)	p. 6
8.1 訊息格式 (複習)	p. 6
8.2 伺服器：每兩秒推一次時間	p. 6
8.3 用戶端	p. 7
9. WebSocket	p. 9
9.1 啟用 WebSockets	p. 9
9.2 回音程式 (echo)	p. 9
9.3 簡易聊天室	p. 10
9.4 共用塗鴉板	p. 12
9.5 SignalR 補充	p. 12
10. AJAX 上傳檔案	p. 14
10.1 IFormFile 與 multer 對照	p. 14
10.2 單檔上傳：個人資料 + 大頭貼	p. 14
10.3 多檔上傳：相簿	p. 15
10.4 注意事項	p. 16
附錄：Node ↔ .NET 常用對照	p. 17

前言

適用對象

- 已熟悉 ASP.NET Core 基本概念：路由、middleware、async/await、依賴注入
- 想學 HTML5 用戶端 API，但不想為了搭舞台另外學 Node.js
- 對 `IFormFile`、`HttpContext`、`Task`、`CancellationToken` 不陌生

技術棧選擇

項目	本文選擇	替代方案
.NET 版本	.NET 10 LTS (2025-11 釋出)	.NET 8 LTS (2026-11 EOL，差異主要在 SSE 簡化 API)
API 風格	Minimal API (程式碼最少)	Controller-based (適合大型專案)
WebSocket	原生 <code>System.Net.WebSockets</code>	SignalR (高階抽象，§9.5 補充說明)
JSON	<code>System.Text.Json</code> (內建)	Newtonsoft.Json

.NET 9 STS 已於 2026-05 結束支援。教學環境推薦直接用 .NET 10 LTS。
若必須用 .NET 8，文中 `TypedResults.ServerSentEvents` 寫法 (§8.2 簡潔版) 改回手動寫 header + `WriteAsync` 即可，行為等價。

環境準備

確認 SDK：

```
dotnet --version # 應為 10.x.x
```

沒有的話從 <https://dotnet.microsoft.com/download> 下載安裝 (macOS 選 Arm64 安裝包)。

方式 A：用既有範例專案 (推薦)

```
git clone https://github.com/shinder/html5-js-api-examples
cd html5-js-api-examples/csharp-backend
dotnet watch run # 修改原始碼自動重啟
```

開 <http://localhost:3031> 會看到 `../public/` 的檔案列表 (對應 Node 的 `serve-index`)。所有 §8 / §9 / §10 端點都已寫好。

專案結構：

```
html5-js-api-examples/
├─ public/           ← 前端 HTML/JS (兩個後端共用同一份)
├─ index.js          ← Node 後端入口
├─ routes/           ← Node WS handlers
└─ csharp-backend/   ← .NET 後端 (本文件對應)
    └─ Program.cs
```

```
└─ Html5CsharpBackend.csproj
└─ Properties/launchSettings.json
```

同 port、不能同時跑：Node 與 .NET 都聽 localhost:3031。要切換後端先停掉一邊。

方式 B：從零建立自己的專案

```
mkdir my-backend && cd my-backend
dotnet new web
dotnet watch run
```

Program.cs 最小骨架：

```
using Microsoft.Extensions.FileProviders;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDirectoryBrowser(); // 對應 Node 的 serve-index
var app = builder.Build();

// 把 ../public/ 當靜態檔來源（與 Node 共用同一份前端）
var publicPath = Path.GetFullPath(
    Path.Combine(builder.Environment.ContentRootPath, "..", "public"));
var fileProvider = new PhysicalFileProvider(publicPath);
app.UseDefaultFiles(new DefaultFilesOptions { FileProvider = fileProvider });
app.UseStaticFiles(new StaticFileOptions { FileProvider = fileProvider });
app.UseDirectoryBrowser(new DirectoryBrowserOptions { FileProvider =
    fileProvider });

app.UseWebSockets(); // $9 需要，啟用 WebSocket middleware

app.MapGet("/", () => "Hello"); // 將被靜態檔目錄列表覆蓋

app.Run("http://localhost:3031");
```

檔案清單順序：內建的目錄瀏覽會照檔案系統回傳的順序列出，不會自己排序。macOS 的 APFS 不保證順序，清單看起來會是亂的；Windows 的 NTFS 通常已照檔名排好。這不是程式寫錯，只是平台差異。想固定排序，繼承 `HtmlDirectoryFormatter` 覆寫 `GenerateContentAsync`，把清單排好再交給父類別輸出，然後在 `DirectoryBrowserOptions` 的 `Formatter` 指定它：

```
using System.Text.Encodings.Web;
using Microsoft.AspNetCore.StaticFiles;

// 資料夾在前，同類再依檔名排序（不分大小寫），與 Node 的 serve-index 行為一致
sealed class SortedDirectoryFormatter(HtmlEncoder encoder) :
```

```

HtmlDirectoryFormatter(encoder)
{
    public override Task GenerateContentAsync(HttpContext context,
        IEnumerable<FileInfo> contents)
    {
        var sorted = contents
            .OrderByDescending(f => f.IsDirectory)
            .ThenBy(f => f.Name, StringComparer.OrdinalIgnoreCase);
        return base.GenerateContentAsync(context, sorted);
    }
}

// 在 UseDirectoryBrowser 指定：
// app.UseDirectoryBrowser(new DirectoryBrowserOptions
// {
//     FileProvider = fileProvider,
//     Formatter = new SortedDirectoryFormatter(HtmlEncoder.Default),
// });

```

範例專案 `csharp-backend/Program.cs` 已經這樣做。注意 Minimal API 的 `Program.cs` 是 top-level statements，類別宣告要放在檔案最後面。

設 `Properties/launchSettings.json` 指定 port、不開瀏覽器：

```

{
  "profiles": {
    "http": {
      "commandName": "Project",
      "launchBrowser": false,
      "applicationUrl": "http://localhost:3031",
      "environmentVariables": { "ASPNETCORE_ENVIRONMENT": "Development" }
    }
  }
}

```

之後 §8 / §9 / §10 各節的程式碼往 `Program.cs` 加進去即可。

CORS 提醒：開發時前端與後端跑在同一個 server（`localhost:3031`）就沒 CORS 問題。如果分開跑（例如 VS Code Live Server `5500` + ASP.NET `3031`），要加 `app.UseCors(...)`。

與原文件 Express 的最大差異

項目	Express	ASP.NET Core
寫串流	<code>res.write(data)</code> 自動 flush	<code>Response.WriteAsync</code> 後要 <code>FlushAsync()</code> ，否則被緩衝
用戶端關閉	<code>req.on("close")</code>	<code>HttpContext.RequestAborted</code> (<code>CancellationToken</code>)
Body 解析	中介軟體 <code>express.json()</code>	<code>[FromBody]</code> T 模型繫結自動處理
檔案上傳	中介軟體 <code>multer</code>	內建 <code>IFormFile</code> / <code>IFormFileCollection</code>
連線清單	<code>wsServer.clients</code> (Set)	自己維護 <code>ConcurrentDictionary</code>

8. Server-Sent Event (SSE)

觀念、訊息格式、用戶端 `EventSource` 用法請見原文件 §8。本節只示範 ASP.NET Core 的伺服器端實作。

8.1 訊息格式（複習）

伺服器端必須回的標頭：

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

訊息以 欄位：值\n 表示，事件之間用空行（\n\n）分隔：

```
id: 20\n
data: 傳送的資料\n\n
```

8.2 伺服器端：每兩秒推一次時間

手動寫法（任何版本通用）

```
// Program.cs
app.MapGet("/try-sse", async (HttpContext ctx) =>
{
    ctx.Response.Headers["Content-Type"] = "text/event-stream";
    ctx.Response.Headers["Cache-Control"] = "no-cache";
    ctx.Response.Headers["Connection"] = "keep-alive";

    var token = ctx.RequestAborted; // 用戶端關閉時 token 自動取消
    var id = 30;

    while (!token.IsCancellationRequested)
    {
        var now = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");
        await ctx.Response.WriteAsync($"id: {id++}\n", token);
        await ctx.Response.WriteAsync($"data: {now}\n\n", token);
        await ctx.Response.Body.FlushAsync(token);

        try { await Task.Delay(2000, token); }
        catch (TaskCanceledException) { break; }
    }
});
```

幾個重點：

- `HttpContext.RequestAborted` 是 ASP.NET 提供的 `CancellationToken`，對應 Node 的 `req.on("close")`：用戶端關閉分頁時 token 被觸發、`Task.Delay` 拋 `TaskCanceledException`、迴圈結束。

- 必須 `FlushAsync()` 訊息才會立刻送出。ASP.NET Core 預設會緩衝寫出的內容，沒 flush 用戶端就看不到串流。Node Express 沒這問題。

簡潔寫法 (.NET 10)

.NET 10 內建 `TypedResults.ServerSentEvents` 工廠，把「寫 header、迴圈、flush、cancellation」全包好。我們只負責產出 `SseItem<T>`：

```
using System.Net.ServerSentEvents; // SseItem<T>
using System.Runtime.CompilerServices; // [EnumeratorCancellation]

app.MapGet("/try-sse",
    (Cancellation token) =>
    TypedResults.ServerSentEvents(GenerateEvents(token)));

static async IAsyncEnumerable<SseItem<string>> GenerateEvents(
    [EnumeratorCancellation] Cancellation token)
{
    var id = 30;
    while (!token.IsCancellationRequested)
    {
        var now = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");
        yield return new SseItem<string>(now) { EventId = (id++).ToString() };

        try { await Task.Delay(2000, token); }
        catch (TaskCanceledException) { yield break; }
    }
}
```

對比手動寫法的優點：

- 不必設 header、不必呼叫 `WriteAsync` / `FlushAsync` — 框架包好
- `Cancellation token` 從 minimal API 參數自動注入，不必透過 `HttpContext.RequestAborted`
- `IAsyncEnumerable<T>` 是 producer/consumer 模式，邏輯與框架解耦，可單獨測試 `GenerateEvents`
- `SseItem<T>` 還支援 `EventType`（event：欄位）與自訂 retry 時間

`SseItem<T>` 的常用欄位：

屬性	對應 SSE 欄位	說明
<code>Data</code>	<code>data:</code>	主資料；建構子第一參數
<code>EventId</code>	<code>id:</code>	訊息 id，用戶端會記住給重連用
<code>EventType</code>	<code>event:</code>	自訂事件名，預設 <code>message</code>
<code>ReconnectionIn</code>	<code>retry:</code>	連線中斷後重試的毫秒數

8.3 用戶端

前端程式碼跟原文件 §8.3 完全一樣（`new EventSource("/try-sse")` 接 message 事件），請直接用 `901-sse.html`。

與 Express 對照的最大踩雷：Node 的 `res.write()` 立刻送出；ASP.NET Core 預設會緩衝，必須顯式 `FlushAsync()`。這是兩邊伺服器最常被卡住的差異。

9. WebSocket

觀念與用戶端 API (`open` / `message` / `close` / `error` 事件、`readyState`) 請見原文件 §9.1。本節示範三個範例 (回音、聊天室、共用塗鴉板) 的 ASP.NET Core 伺服器端，並在最後簡介 SignalR。

9.1 啟用 WebSockets

ASP.NET Core 處理 WebSocket 要：

1. 在 pipeline 中加入 `app.UseWebSockets()`
2. 端點裡檢查 `ctx.WebSockets.IsWebSocketRequest`
3. 用 `AcceptWebSocketAsync()` 把連線升級成 WebSocket

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseWebSockets(); // 必加
app.UseStaticFiles();
```

9.2 回音程式 (echo)

對應原文件 `routes/ws-echo.js`：用戶端送什麼，伺服器原樣回。

```
using System.Net.WebSockets;
using System.Text;

app.Map("/ws/echo", async (HttpContext ctx) =>
{
    if (!ctx.WebSockets.IsWebSocketRequest)
    {
        ctx.Response.StatusCode = 400;
        return;
    }

    using var ws = await ctx.WebSockets.AcceptWebSocketAsync();
    var ip = ctx.Connection.RemoteIpAddress?.ToString();

    await SendText(ws, $"連線了，你來自 {ip}");

    var buffer = new byte[1024 * 4];
    while (ws.State == WebSocketState.Open)
    {
        var result = await ws.ReceiveAsync(buffer, CancellationToken.None);
        if (result.MessageType == WebSocketMessageType.Close)
        {
            await ws.CloseAsync(WebSocketCloseStatus.NormalClosure, null,
                CancellationToken.None);
            break;
        }
    }
}
```

```

    }
    var text = Encoding.UTF8.GetString(buffer, 0, result.Count);
    await SendText(ws, text); // 原訊息送回
  }
});

static async Task SendText(WebSocket ws, string text)
{
    var bytes = Encoding.UTF8.GetBytes(text);
    await ws.SendAsync(bytes, WebSocketMessageType.Text,
        endOfMessage: true, CancellationToken.None);
}

```

對照觀念：

Node ws	ASP.NET 原生
<code>ws.send(string)</code>	<code>SendAsync(bytes, ...)</code> ，要自己編 UTF-8
<code>ws.on("message", ...)</code>	迴圈呼叫 <code>ReceiveAsync()</code>
<code>wsServer.clients</code>	自己維護 <code>ConcurrentDictionary</code> 或 <code>List</code> + <code>lock</code>
Server 自動跑	每條連線就是一個 <code>async loop</code> (thread pool)

前端 `A03-WebSocket-echo.html` 用 `ws://${location.host}/ws/echo` 一跟 Node 後端版本完全相同的 URL。換後端不必改前端。

原文件 §9.3 的 Node 版是在 `upgrade` 事件裡用 `path` 查表分流，因此要防 `Object.prototype` 上的成員被查到；.NET 版每條 `path` 各自 `app.Map(...)`，由路由系統比對，沒有這個問題。

9.3 簡易聊天室

對應 `routes/ws-chat.js`：第一次傳入的訊息當名字、之後每則訊息廣播給所有連線。

```

using System.Collections.Concurrent;
using System.Net.WebSockets;
using System.Text;

var clients = new ConcurrentDictionary<string, WebSocket>();

app.Map("/ws/chat", async (HttpContext ctx) =>
{
    if (!ctx.WebSockets.IsWebSocketRequest) { ctx.Response.StatusCode = 400;
    return; }

    using var ws = await ctx.WebSockets.AcceptWebSocketAsync();
    var id = Guid.NewGuid().ToString();
    clients[id] = ws;

    var ip = ctx.Connection.RemoteIpAddress?.ToString();
    string? userName = null;

```

```

var buffer = new byte[1024 * 4];

try
{
    while (ws.State == WebSocketState.Open)
    {
        var result = await ws.ReceiveAsync(buffer, CancellationToken.None);
        if (result.MessageType == WebSocketMessageType.Close) break;

        var text = Encoding.UTF8.GetString(buffer, 0, result.Count);

        if (userName is null)
        {
            userName = text;
            await Broadcast($"{userName}({ip}) 進入，目前 {clients.Count} 人");
        }
        else
        {
            await Broadcast($"{userName}: {text}");
        }
    }
}
finally
{
    clients.TryRemove(id, out _);
    if (userName is not null)
        await Broadcast($"{userName}({ip}) 離開，目前 {clients.Count} 人");
}
});

async Task Broadcast(string message)
{
    var bytes = Encoding.UTF8.GetBytes(message);
    foreach (var client in clients.Values)
    {
        if (client.State != WebSocketState.Open) continue;
        try
        {
            await client.SendAsync(bytes, WebSocketMessageType.Text, true,
CancellationTokens.None);
        }
        catch { /* 連線失敗就略過，下次廣播會排除 */ }
    }
}

```

幾個對照重點：

- Node 的 `wsServer.clients` 是內建的 `Set`；.NET 自己維護 `ConcurrentDictionary<string, WebSocket>`，key 用 `Guid` 當連線 id 方便追蹤。
- `try/finally` 保證連線中斷時從清單移除（對應 Node `ws.on("close")`）。
- `c.send(msg)` 要先 `Encoding.UTF8.GetBytes()` 把字串轉 byte。

9.4 共用塗鴉板

對應 `routes/ws-draw.js` — 收到訊息廣播原文，不做任何加工：

```
var drawClients = new ConcurrentDictionary<string, WebSocket>();

app.Map("/ws/draw", async (HttpContext ctx) =>
{
    if (!ctx.WebSockets.IsWebSocketRequest) { ctx.Response.StatusCode = 400;
return; }

    using var ws = await ctx.WebSockets.AcceptWebSocketAsync();
    var id = Guid.NewGuid().ToString();
    drawClients[id] = ws;

    var buffer = new byte[1024 * 4];
    try
    {
        while (ws.State == WebSocketState.Open)
        {
            var result = await ws.ReceiveAsync(buffer, CancellationToken.None);
            if (result.MessageType == WebSocketMessageType.Close) break;

            var payload = buffer.AsMemory(0, result.Count);
            foreach (var c in drawClients.Values)
            {
                if (c.State != WebSocketState.Open) continue;
                try
                {
                    await c.SendAsync(payload, WebSocketMessageType.Text,
                        true, CancellationToken.None);
                }
                catch { }
            }
        }
    }
    finally
    {
        drawClients.TryRemove(id, out _);
    }
});
```

前端 `A05-WebSocket-draw.html` 用 `ws://${location.host}/ws/draw` — 跟 Node 後端版相同。

9.5 SignalR 補充

ASP.NET Core 還提供 SignalR 這個高階抽象，把上面那一堆連線管理、訊息序列化都包好：

```
public class ChatHub : Hub
{
```

```

    public async Task SendMessage(string user, string message)
    {
        await Clients.All.SendAsync("ReceiveMessage", user, message);
    }
}

// Program.cs
builder.Services.AddSignalR();
app.MapHub<ChatHub>("/chatHub");

```

比較	原生 WebSocket	SignalR
連線管理	自己維護清單	內建 (含 Group、Connection ID)
自動重連	自己處理	用戶端 SDK 內建
降級	沒有	自動降級為 long polling、SSE
多 server	自己同步	接 Redis backplane 自動同步
用戶端	純 WebSocket 物件	必須用 SignalR client (@microsoft/signalr)
跟講義對應	與 Node ws 直接對等	用戶端 HTML 要改寫，與本講義不直接相容

結論：教學階段用原生 WebSocket（與 Node 對等、用戶端共用講義 HTML）；要上正式生產系統可以考慮 SignalR。

10. AJAX 上傳檔案

觀念、`FormData`、用戶端 `fetch` 用法請見原文件 §10。本節以 ASP.NET Core 寫單檔與多檔上傳。

10.1 IFormFile 與 multer 對照

Node multer	ASP.NET Core
<code>upload.single("avatar")</code>	參數 <code>IFormFile? avatar</code>
<code>upload.array("photos")</code>	參數 <code>IFormFileCollection photos</code>
文字欄位 <code>req.body.user</code>	<code>[FromForm] string user</code>
<code>req.file.filename</code>	自己決定 (從 <code>IFormFile.FileName</code> 推或產隨機)
<code>multer.diskStorage</code>	用 <code>IFormFile.CopyToAsync(stream)</code> 自己寫
<code>fileFilter</code>	自己檢查 <code>IFormFile.ContentType</code>

ASP.NET Core 沒有 multer 那種「設定一次到處用」的中介，但 `IFormFile` 的解析是內建的，寫檔邏輯獨立寫一段就好，整體程式碼量不會多太多。

10.2 單檔上傳：個人資料 + 大頭貼

對應原文件 `B01-upload-single.html` 的後端：

```
using System.Text.Json;

// MIME → 副檔名 (同時也是白名單)
var extMap = new Dictionary<string, string>
{
    ["image/jpeg"] = ".jpg",
    ["image/png"] = ".png",
    ["image/webp"] = ".webp",
};

app.MapPost("/uploads/profile", async (
    [FromForm] string? user,
    [FromForm] string? description,
    IFormFile? avatar) =>
{
    var profilePath = Path.Combine(publicPath, "profile.json");

    // 讀取既有資料 (不存在就用空 dict)
    var data = new Dictionary<string, string?>();
    if (File.Exists(profilePath))
    {
        try
        {
            var json = await File.ReadAllTextAsync(profilePath);
            data = JsonSerializer.Deserialize<Dictionary<string, string?>>(json) ??
new();

```

```

    }
    catch { /* 檔案壞了當空 */ }
}

// 合併新資料
if (user is not null) data["user"] = user;
if (description is not null) data["description"] = description;

// 處理上傳檔案
if (avatar is not null && extMap.TryGetValue(avatar.ContentType, out var ext))
{
    var imagesDir = Path.Combine(publicPath, "images");
    Directory.CreateDirectory(imagesDir);
    var fileName = $"{Guid.NewGuid()}{ext}";

    await using var stream = File.Create(Path.Combine(imagesDir, fileName));
    await avatar.CopyToAsync(stream);

    data["avatar"] = $"/images/{fileName}";
}

try
{
    await File.WriteAllTextAsync(profilePath, JsonSerializer.Serialize(data));
    return Results.Ok(new { success = true, data });
}
catch
{
    return Results.Ok(new { success = false, data });
}
}).DisableAntiforgery();

```

幾個細節：

- **[FromForm]** 告訴模型繫結讀 multipart form 欄位（對應 Node 的 `req.body`）。
- **IFormFile** 是內建型別，自帶 `ContentType`、`FileName`、`Length`，等同於 multer 的 `req.file`。
- **.DisableAntiforgery()** — Minimal API 對 form post 預設啟用 antiforgery (CSRF) token 檢查，前端 fetch 沒附 token 會 400。教學情境直接 disable 較簡單；正式應用要用 `IAntiforgery` 服務正確處理。
- **Guid.NewGuid().ToString()** 等價於 Node 的 `uuid.v4()`。
- **Path.Combine** 比直接字串相加跨平台，永遠選它。
- **publicPath** 是 §「環境準備」設定的 `../public/` 絕對路徑，與前端共用 — 上傳的圖檔放 `public/images/`，前端就可以用 `/images/xxx.png` 讀到。如果是獨立專案放 `wwwroot/`，把 `publicPath` 換成 `Path.Combine(builder.Environment.WebRootPath, ...)`。

10.3 多檔上傳：相簿

對應 `B02-upload-multiple.html`：

```
app.MapPost("/uploads/photos", async (IFormFileCollection photos) =>
{
    var imagesDir = Path.Combine(publicPath, "images");
    Directory.CreateDirectory(imagesDir);

    var urls = new List<string>();
    foreach (var photo in photos)
    {
        if (!extMap.TryGetValue(photo.ContentType, out var ext)) continue;

        var fileName = $"{Guid.NewGuid()}_{ext}";
        var savePath = Path.Combine(imagesDir, fileName);

        await using var stream = File.Create(savePath);
        await photo.CopyToAsync(stream);

        urls.Add($"images/{fileName}");
    }
    return Results.Ok(urls);
}).DisableAntiforgery();
```

前端送出的 `FormData` 跟 Node 版本完全一樣（`name="photos"` 多檔），不需要任何前端調整。
`IFormFileCollection` 自動把同名 form field 收成集合。

10.4 注意事項

- **檔案大小限制**：ASP.NET Core 預設限制單一請求 30 MB，要更大需要 `[RequestSizeLimit]` 屬性或 server 設定（Kestrel：`MaxRequestBodySize`）。
- **白名單檢查不可信**：`avatar.ContentType` 是瀏覽器送來的、可以偽造。正式環境要看實際檔案開頭 byte（magic bytes）判定真實型別。
- **不在白名單是靜默略過**：與 Node 版 multer 的 `fileFilter` 一樣，型別不符的檔案不會存、也不會回錯誤，前端看到的仍是成功。要讓使用者知道被拒，改回 `Results.BadRequest(...)`。
- **路徑安全**：永遠用 `Guid.NewGuid()` 自己決定檔名，不要直接用 `IFormFile.FileName` — 用戶端可塞 `../etc/passwd` 之類的路徑做 directory traversal 攻擊。
- **同步寫檔阻塞 thread pool**：高併發時用 `await CopyToAsync` 而非 `CopyTo`（這份範例已是 async）。

附錄：Node ↔ .NET 常用對照

針對講義會用到的 server 端 API，快速對照：

概念	Node / Express	ASP.NET Core
起 server	<code>app.listen(3031)</code>	<code>app.Run("http://localhost:3031")</code>
GET 路由	<code>app.get("/x", handler)</code>	<code>app.MapGet("/x", handler)</code>
POST 路由	<code>app.post("/x", handler)</code>	<code>app.MapPost("/x", handler)</code>
URL params	<code>req.params.id</code>	<code>(string id) => ...</code> (從 route template 自動繫結)
Query string	<code>req.query</code>	<code>[FromQuery] string x /</code> <code>HttpRequest.Query</code>
Body (JSON)	<code>req.body</code> (加 <code>express.json()</code>)	<code>[FromBody] T param</code> 自動 deserialize
Body (form)	<code>req.body</code> (加 <code>express.urlencoded()</code>)	<code>[FromForm] T param</code>
檔案上傳	<code>multer + req.file</code>	<code>IFormFile</code> 參數
回 JSON	<code>res.json(obj)</code>	<code>Results.Ok(obj)</code> 或直接 <code>return obj</code>
設 status code	<code>res.status(400).json(...)</code>	<code>Results.BadRequest(obj)</code>
寫串流	<code>res.write(data)</code>	<code>await Response.WriteAsync(data) +</code> <code>FlushAsync()</code>
監聽用戶端關閉	<code>req.on("close", ...)</code>	<code>HttpContext.RequestAborted</code> (<code>CancellationToken</code>)
Static files	<code>app.use(express.static("public"))</code>	<code>app.UseStaticFiles()</code> (預設 <code>wwwroot/</code>)
WebSocket server	<code>import { WebSocketServer } from "ws"</code>	<code>app.UseWebSockets() +</code> <code>AcceptWebSocketAsync()</code>
廣播給所有連線	<code>wsServer.clients.forEach(...)</code>	自己維護的 <code>ConcurrentDictionary</code> + <code>foreach</code>
環境變數	<code>process.env.X</code>	<code>IConfiguration["X"]</code> 或 <code>Environment.GetEnvironmentVariable("X")</code>
JSON 序列化	<code>JSON.stringify / JSON.parse</code>	<code>JsonSerializer.Serialize / Deserialize</code>
隨機唯一識別	<code>uuid.v4()</code>	<code>Guid.NewGuid()</code>

講義其他章節（純前端 §1—§7、§11、§12，附錄一/二/三）跟後端語言無關，直接用原文件即可。